

Round-Translation Error Analysis and Findings

Overview

This document analyzes translation errors from our from-scratch transformer model performing round-translation (English -> Dutch -> English) on a Kitchen Nightmares episode transcript. We evaluated 150 sentences, scoring each translation and identifying error patterns.

Scoring Distribution

Translation Quality Breakdown:

- Score 3 (Correct): 36 sentences (24%)
- Score 2 (Minor errors): 48 sentences (32%)
- Score 1 (Incorrect): 66 sentences (44%)

The results show that less than a quarter of translations were fully accurate. This highlights the challenges of building a machine translation model from scratch with limited resources, but also demonstrates that the model learned meaningful translation patterns since it performed far better than random chance.

Main Error Categories

Through our analysis, we identified 6 distinct types of errors:

1. Missing Vocabulary
 - The model replaced unknown words with [UNK] tokens. This was our most frequent error type.
 - Examples:
 - Restaurant terms: waitress, tiramisu, chewier
 - Business vocabulary: collateral, finance, retirement
 - People and places: Tatiana, Casamero, La Verne
 - Our model trained on movie subtitles with a vocabulary of only 10,000 words. When the model encountered words outside of the scope of this vocabulary, it had no choice but to mark them as unknown.
2. Wrong Word Choice
 - The model picked words that were related in meaning but not quite right.
 - Examples:
 - opportunity -> chance
 - scared -> afraid
 - get -> have
 - he's pretty awful -> and he's beautiful
 - The model learned that certain words appear in similar contexts. Without understanding actual meaning, it sometimes picked the wrong option from semantically related words.
3. Problems with Names
 - The model completely failed with proper names.
 - Examples:
 - I'm Val -> I'm dying

- Tatiana -> [UNK]
 - Caprice -> [UNK]
- Names should never be translated, but stay the same across languages, however, our model didn't know what a name was and what wasn't.
- 4. Sentence Structure Problems
 - Even when the model knew the words, it sometimes put them in the wrong order or left parts out.
 - Examples:
 - I just wanted to keep it -> I wanted to keep it (lost "just")
 - It is very serious -> It's serious (lost "very")
 - Glass of wine in her hand -> glass of her hand (lost "wine")
 - I've loved this restaurant -> I love this restaurant (lost tense)
 - Why this happened:
 - English and Dutch have different word order rules.
 - English has many more tense forms than Dutch
 - The model learned common patterns but struggled when multiple grammar rules applied.
- 5. Idioms Translated Literally
 - The model translated English idioms word-for-word.
 - Examples:
 - Talking out of your ass -> you talk to your ass
 - Play the blame game -> they're playing it
- 6. Lost Fillers and Conversational Elements
 - Spoken language includes word like "um", "oh", and "like" that the model dropped often or mistranslated.
 - Examples:
 - um, oh -> completely omitted
 - oh, okay -> okay
 - mm-hmm -> nice?
 - Movie subtitles are often cleaned-up versions of speech where filler words are often removed to save space.

Patterns

Error Propagation

We tracked what happened to sentences as they moved through the complete round-translation process from English to Dutch and back to English. The results showed clear patterns in how translation quality either persisted or degraded through multiple steps.

Sentences that were translated correctly in the first step usually stayed correct. For example, "How are you?" became "hoe gaat het?" and return as "how are you?" without any problems. The model's understanding of these common phrases was strong enough to survive both translation steps.

On the other hand, badly translated sentences almost never fixed themselves. The "I'm Val" example shows this clearly – the model thought "Val" was the Dutch word for "fall", created "ik val dood", which came back as "I'm dying". The first mistake led to a bigger problem in the second step.

The main takeaway is that errors multiply through the round-translation process instead of averaging out. When the first translation makes a mistake, the second translation works with bad input and compounds the problem.

Domain Mismatch

Our training data came from movie subtitles – conversations from dramas, comedies, and action films. This diverse dataset was chosen on purpose to build a model that could generalize across different speaking styles and topics. The Kitchen Nightmares transcript tested this generalization by introducing specialized restaurant language. Common conversational patterns like greetings and questions worked regardless of context. This shows that training on diverse movie dialogue successfully captured fundamental language patterns that transfer across domains.

However, restaurant terms, business vocabulary, and reality TV elements fell outside our training scope. The positive finding is that our movie-trained model handled grammatical and conversational aspects of the restaurant transcript well.

What the Model Did Well

The model showed clear strengths in certain areas. Very short phrases, common expressions, everyday phrases, family words, and small function words consistently gave good results. Basic grammar succeeded when it was common in training. Present tense verbs in simple sentences, question formation, and negation all gave reliable results as well.

The model did great with frequent and simple patterns it saw many times during training. It struggled with rare words, complex grammar, and specialized topics.

Comparison with Pretrained Models

Our from-scratch models achieved 18.77 BLEU with 24% of sentences fully correct. We trained on 100,000 sentences pairs with a 10,000-word vocabulary. The pretrained Helsinki-NLP MarianMT model achieved 27.31 BLEU. It was trained on millions of diverse sentences pairs with 50,000-word vocabulary.

What We Learned

Our most important discovery was that vocabulary coverage matters more than almost anything else. Thirty-five percent of all errors came from words the model simply didn't know. Named entities need special handling, for example, professional systems use Named Entity Recognition to identify and preserve names unchanged through translation.

During the iterating process, optimization settings proved surprisingly important. Increasing batch size from 32 to 128 gave us the largest improvement. This taught us that how we train a model can matter as much as what we train. We also learned that due to round-translation, errors pile up rather than cancel out. Significant amount of the bad translations stayed bad or got worse. Across all tests, one pattern stood out - the model did well on common patterns but struggled with the rare ones.

Pretrained Models for Production

Building from scratch showed we understand how transformers work and how training affects final quality. We improved performance from 12.5 to 18.8 BLEU through experiments with batch

sizes, vocabulary expansion, and extended training. However, for our actual NLP pipeline, we use the pretrained Helsinki-NLP MarianMT model instead of our custom model. This choice reflects a big quality difference between educational models and production systems.